

SGD#

<https://pytorch.org/docs/stable/generated/torch.optim.SGD.html#torch.optim.SGD>

Description:

Implements stochastic gradient descent (optionally with momentum). Nesterov momentum is based on the formula from On the importance of initialization and momentum in deep learning. `params` (iterable) – iterable of parameters or `named_parameters` to optimize or iterable of dicts defining parameter groups. When using `named_parameters`, all parameters in all groups should be named `lr` (float, Tensor, optional) – learning rate (default: 1e-3) `momentum` (float, optional) – momentum factor (default: 0)

Function Signature

```
class torch.optim.SGD(params, lr=0.001, momentum=0,
dampening=0, weight_decay=0, nesterov=False, *,
maximize=False, foreach=None, differentiable=False,
fused=None)[source]#
```

Parameters

```
add_param_group(param_group)[source]#
```

Parameters

```
load_state_dict(state_dict)[source]#
```

Returns:

dict[str, Any]

Code Examples

```
>>> optimizer = torch.optim.SGD(model.parameters(), lr=0.1,
momentum=0.9)
>>> optimizer.zero_grad()
>>> loss_fn(model(input), target).backward()
>>> optimizer.step()
```

```
>>> model = torch.nn.Linear(10, 10)
>>> optim = torch.optim.SGD(model.parameters(), lr=3e-4)
>>> scheduler1 = torch.optim.lr_scheduler.LinearLR(
...     optim,
...     start_factor=0.1,
...     end_factor=1,
...     total_iters=20,
... )
>>> scheduler2 = torch.optim.lr_scheduler.CosineAnnealingLR(
...     optim,
...     T_max=80,
...     eta_min=3e-5,
... )
>>> lr = torch.optim.lr_scheduler.SequentialLR(
...     optim,
...     schedulers=[scheduler1, scheduler2],
...     milestones=[20],
... )
>>> lr.load_state_dict(torch.load("./save_seq.pt"))
>>> # now load the optimizer checkpoint after loading the
LRScheduler
>>> optim.load_state_dict(torch.load("./save_optim.pt"))
```

```
hook(optimizer) -> None
```

```
hook(optimizer, state_dict) -> state_dict or None
```

```
hook(optimizer, state_dict) -> state_dict or None
```

```
hook(optimizer) -> None
```

```
hook(optimizer, args, kwargs) -> None
```

```
hook(optimizer, args, kwargs) -> None or modified args and  
kwargs
```

```

{
  'state': {
    0: {'momentum_buffer': tensor(...), ...},
    1: {'momentum_buffer': tensor(...), ...},
    2: {'momentum_buffer': tensor(...), ...},
    3: {'momentum_buffer': tensor(...), ...}
  },
  'param_groups': [
    {
      'lr': 0.01,
      'weight_decay': 0,
      ...
      'params': [0]
      'param_names' ['param0'] (optional)
    },
    {
      'lr': 0.001,
      'weight_decay': 0.5,
      ...
      'params': [1, 2, 3]
      'param_names': ['param1', 'layer.weight',
'layer.bias'] (optional)
    }
  ]
}

```

Notes & Warnings

NOTE The foreach and fused implementations are typically faster than the for-loop, single-tensor implementation, with fused being theoretically fastest with both vertical and horizontal fusion. As such, if the user has not specified either flag (i.e., when `foreach = fused = None`), we will attempt defaulting to the foreach implementation when the tensors are all on CUDA. Why not fused? Since the fused implementation is relatively new, we want to give it sufficient bake-in time. To specify fused, pass `True` for fused. To force running the for-loop implementation, pass `False` for either foreach or fused.

NOTE The implementation of SGD with Momentum/Nesterov subtly differs from Sutskever et al. and implementations in some other frameworks. Considering the specific case of Momentum, the update can be written as
$$v_{t+1} = \mu * v_t + g_{t+1}, p_{t+1} = p_t - lr * v_{t+1}$$

$$v_{t+1} = \mu * v_t + lr * g_{t+1}, p_{t+1} = p_t - v_{t+1}$$
 where p , g , v and μ denote the parameters, gradient, velocity, and momentum respectively. This is in contrast to Sutskever et al. and other frameworks which employ an update of the form
$$v_{t+1} = \mu * v_t + lr * g_{t+1}, p_{t+1} = p_t - v_t$$

$$v_{t+1} = \mu * v_t + g_{t+1}, p_{t+1} = p_t - v_{t+1}$$
 The Nesterov vers

⚠ WARNING Make sure this method is called after initializing `torch.optim.lr_scheduler.LRScheduler`, as calling it beforehand will overwrite the loaded learning rates.

NOTE

Note The names of the parameters (if they exist under the “param_names” key of each param group in state_dict()) will not affect the loading process. To use the parameters’ names for custom cases (such as when the parameters in the loaded state dict differ from those initialized in the optimizer), a custom register_load_state_dict_pre_hook should be implemented to adapt the loaded dict accordingly. If param_names exist in loaded state dict param_groups they will be saved and override the current names, if present, in the optimizer state. If they do not exist in loaded state dict, the optimizer param_names will remain unchanged.

Detailed Documentation

Documentation

Implements stochastic gradient descent (optionally with momentum).

Nesterov momentum is based on the formula from On the importance of initialization and momentum in deep learning.

params (iterable) – iterable of parameters or named_parameters to optimize or iterable of dicts defining parameter groups. When using named_parameters, all parameters in all groups should be named

lr (float, Tensor, optional) – learning rate (default: 1e-3)

momentum (float, optional) – momentum factor (default: 0)

dampening (float, optional) – dampening for momentum (default: 0)

`weight_decay` (float, optional) – weight decay (L2 penalty) (default: 0)

`nesterov` (bool, optional) – enables Nesterov momentum. Only applicable when momentum is non-zero. (default: False)

`maximize` (bool, optional) – maximize the objective with respect to the params, instead of minimizing (default: False)

`foreach` (bool, optional) – whether foreach implementation of optimizer is used. If unspecified by the user (so `foreach` is `None`), we will try to use foreach over the for-loop implementation on CUDA, since it is usually significantly more performant. Note that the foreach implementation uses $\sim \text{sizeof}(\text{params})$ more peak memory than the for-loop version due to the intermediates being a `tensorlist` vs just one tensor. If memory is prohibitive, batch fewer parameters through the optimizer at a time or switch this flag to `False` (default: `None`)

`differentiable` (bool, optional) – whether autograd should occur through the optimizer step in training. Otherwise, the `step()` function runs in a `torch.no_grad()` context. Setting to `True` can impair performance, so leave it `False` if you don't intend to run autograd through this instance (default: `False`)

`fused` (bool, optional) – whether the fused implementation is used. Currently, `torch.float64`, `torch.float32`, `torch.float16`, and `torch.bfloat16` are supported. (default: `None`)

The `foreach` and `fused` implementations are typically faster than the for-loop, single-tensor implementation, with `fused` being theoretically fastest with both vertical and horizontal fusion. As such, if the user has not specified either flag (i.e., when `foreach` = `fused` = `None`), we will attempt defaulting to the `foreach` implementation when the tensors are all on CUDA. Why not `fused`? Since the `fused` implementation is relatively new, we want to give it sufficient bake-in time. To specify `fused`, pass `True` for `fused`.

To force running the for-loop implementation, pass `False` for either `foreach` or `fused`.

The implementation of SGD with Momentum/Nesterov subtly differs from Sutskever et al. and implementations in some other frameworks.

Considering the specific case of Momentum, the update can be written as

where $\mathbf{p}$, $\mathbf{g}$, $\mathbf{v}$ and μ denote the parameters, gradient, velocity, and momentum respectively.

This is in contrast to Sutskever et al. and other frameworks which employ an update of the form

The Nesterov version is analogously modified.

Moreover, the initial value of the momentum buffer is set to the gradient value at the first step. This is in contrast to some other frameworks that initialize it to all zeros. One notable side effect of this decision is that the first momentum value will not be scaled by dampening. Dampening will be applied starting at the second step.

Add a param group to the Optimizer's `param_groups`.

This can be useful when fine tuning a pre-trained network as frozen layers can be made trainable and added to the Optimizer as training progresses.

`param_group` (dict) – Specifies what Tensors should be optimized along with group specific optimization options.

Load the optimizer state.

`state_dict` (dict) – optimizer state. Should be an object returned from a call to `state_dict()`.

Make sure this method is called after initializing `torch.optim.lr_scheduler.LRScheduler`, as calling it beforehand will overwrite the loaded learning rates.

The names of the parameters (if they exist under the “`param_names`” key of each param group in `state_dict()`) will not affect the loading process. To use the parameters’ names for custom cases (such as when the parameters in the loaded state dict differ from those initialized in the optimizer), a custom `register_load_state_dict_pre_hook` should be implemented to adapt the loaded dict accordingly. If `param_names` exist in loaded state dict `param_groups` they will be saved and override the current names, if present, in the optimizer state. If they do not exist in loaded state dict, the optimizer `param_names` will remain unchanged.

Register a `load_state_dict` post-hook which will be called after `load_state_dict()` is called. It should have the following signature:

The optimizer argument is the optimizer instance being used.

The hook will be called with argument `self` after calling `load_state_dict` on `self`. The registered hook can be used to perform post-processing after `load_state_dict` has loaded the `state_dict`.

`hook (Callable)` – The user defined hook to be registered.

`prepend (bool)` – If `True`, the provided post hook will be fired before all the already registered post-hooks on `load_state_dict`. Otherwise, the provided hook will be fired after all the already registered post-hooks. (default: `False`)

a handle that can be used to remove the added hook by calling `handle.remove()`

`torch.utils.hooks.RemoveableHandle`

Register a `load_state_dict` pre-hook which will be called before `load_state_dict()` is

called. It should have the following signature:

The `optimizer` argument is the optimizer instance being used and the `state_dict` argument is a shallow copy of the `state_dict` the user passed in to `load_state_dict`. The hook may modify the `state_dict` inplace or optionally return a new one. If a `state_dict` is returned, it will be used to be loaded into the optimizer.

The hook will be called with argument `self` and `state_dict` before calling `load_state_dict` on `self`. The registered hook can be used to perform pre-processing before the `load_state_dict` call is made.

`hook` (Callable) – The user defined hook to be registered.

`prepend` (bool) – If True, the provided pre hook will be fired before all the already registered pre-hooks on `load_state_dict`. Otherwise, the provided hook will be fired after all the already registered pre-hooks. (default: False)

a handle that can be used to remove the added hook by calling `handle.remove()`

`torch.utils.hooks.RemoveableHandle`

Register a state dict post-hook which will be called after `state_dict()` is called.

It should have the following signature:

The hook will be called with arguments `self` and `state_dict` after generating a `state_dict` on `self`. The hook may modify the `state_dict` inplace or optionally return a new one. The registered hook can be used to perform post-processing on the `state_dict` before it is returned.

`hook` (Callable) – The user defined hook to be registered.

`prepend` (bool) – If True, the provided post hook will be fired before all the already

registered post-hooks on state_dict. Otherwise, the provided hook will be fired after all the already registered post-hooks. (default: False)

a handle that can be used to remove the added hook by calling handle.remove()

torch.utils.hooks.RemoveableHandle

Register a state dict pre-hook which will be called before state_dict() is called.

It should have the following signature:

The optimizer argument is the optimizer instance being used. The hook will be called with argument self before calling state_dict on self. The registered hook can be used to perform pre-processing before the state_dict call is made.

hook (Callable) – The user defined hook to be registered.

prepend (bool) – If True, the provided pre hook will be fired before all the already registered pre-hooks on state_dict. Otherwise, the provided hook will be fired after all the already registered pre-hooks. (default: False)

a handle that can be used to remove the added hook by calling handle.remove()

torch.utils.hooks.RemoveableHandle

Register an optimizer step post hook which will be called after optimizer step.

It should have the following signature:

The optimizer argument is the optimizer instance being used.

hook (Callable) – The user defined hook to be registered.

a handle that can be used to remove the added hook by calling `handle.remove()`

`torch.utils.hooks.RemovableHandle`

Register an optimizer step pre hook which will be called before optimizer step.

It should have the following signature:

The optimizer argument is the optimizer instance being used. If args and kwargs are modified by the pre-hook, then the transformed values are returned as a tuple containing the `new_args` and `new_kwargs`.

hook (Callable) – The user defined hook to be registered.

a handle that can be used to remove the added hook by calling `handle.remove()`

`torch.utils.hooks.RemovableHandle`

Return the state of the optimizer as a dict.

It contains two entries:

differs between optimizer classes, but some common characteristics hold. For example, state is saved per parameter, and the parameter itself is NOT saved. state is a Dictionary mapping parameter ids to a Dict with state corresponding to each parameter.

parameter group is a Dict. Each parameter group contains metadata specific to the optimizer, such as learning rate and weight decay, as well as a List of parameter IDs of the parameters in the group. If a param group was initialized with `named_parameters()` the names content will also be saved in the state dict.

NOTE: The parameter IDs may look like indices but they are just IDs associating state

with `param_group`. When loading from a `state_dict`, the optimizer will zip the `param_group` params (int IDs) and the optimizer `param_groups` (actual `nn.Parameter`s) in order to match state WITHOUT additional verification.

A returned state dict might look something like:

Perform a single optimization step.

`closure` (Callable, optional) – A closure that reevaluates the model and returns the loss.

Reset the gradients of all optimized `torch.Tensor`s.

`set_to_none` (bool, optional) –

Instead of setting to zero, set the grads to None. Default: True

This will in general have lower memory footprint, and can modestly improve performance. However, it changes certain behaviors. For example:

When the user tries to access a gradient and perform manual ops on it, a None attribute or a Tensor full of 0s will behave differently.

If the user requests `zero_grad(set_to_none=True)` followed by a backward pass, `.grads` are guaranteed to be None for params that did not receive a gradient.

`torch.optim` optimizers have a different behavior if the gradient is 0 or None (in one case it does the step with a gradient of 0 and in the other it skips the step altogether).